PARCIAL 1

Mauricio Hernández González

1= Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de codigo. ¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

La clase es el plano donde se definen los atributos y metodos de cada objeto abstraido dentro del programa.

El objeto es la instancia que se crean en la ejecucion del programa, cada objeto tiene su propio estado con sus respectivos valores.

Mi error en esta pregunta fue el no usar la definicion completa de la clase como el plano y el objeto como una instancia.

2= ¿Qué es la firma de un método y qué partes la forman?

La firma es la forma con la que identificamos un metodo. Esta tiene una estructura estricta donde se forma por el nombre y los tipos de los parametros, en orden, y la respuesta omite los parametros.

3= Compara la anatomia de una variable local y un atributo de instancia: donde se declaran, valor inicial, y cuanto viven.

El atributo de instancia se declara en la clase, su duracion sera igual a la duración que tenga su respectivo objeto, su valor inicial es null.

La variable local se declara dentro del metodo y su duracion de esta variable sera lo que dure el metodo ya que este vive en lo que se ejecuta el metodo, al igual que su valor inicial es null.

4= ¿Qué significa null en una referencia? Explica que ocurre al llamar un método sobre esa referencia

Null en una referencia, significa que la ubicación de la referencia no ha sido encontrada.

Llamar un método de instancia sobre esa referencia truena el codigo debido a que no hay nada a lo que referirse.

5= ¿Cuando es conveniente usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con indice?

for-each es una muy buena opción cuando sola se necesita leer o procesar todo los elementos de principio a fin sin importar la posición exacta.

Conviene usar un recorrido con indice si se busca modificar los elementos del arreglo, si se necesita conocer o mostrar la posición, recorrer al derecho a 'en reversa o al hacer saltos o comparar elementos.

R1 ¿Que imprime?

Escribe la salida exacta, una línea por renglón.

S0
S01
S0111

Ya que la ruta y alias son StringBuilder, se va acumulando S0, S01, S0111.

S21 → Esto se debe a que la linea String texto = "S"+2+1 concatena el texto con 2 y 1. Dejando S21.

3.0 → Esto se debe a que el casting double (7/2) convierte la division en un dato tipo double osea 3.0

R2. ¿Compila, truena o funciona?

Cada inciso va en un main independiente. Indica si compila, si no compila pero truena o funciona y da la causa en una frase.

a)
```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Este codigo no compila, ya que no se hace uso de un casting para convertir el valor double a un valor int. Quebrando el codigo. Esto se soluciona usando un casting (int) para convertir el valor tipo double en un valor tipo int.

b)
```
int[] canales = {2,5,8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Compila pero truena, ya que el codigo "int indice = canales.length;" habla sobre la cantidad de valores dentro del arreglo, mientras que el indice siempre valdra "canales.length - 1" en este caso. Para arreglar este codigo se debe usar el codigo "int indice = canales.length - 1"; para que pueda imprimir el valor de 8.

c)
```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

→ Esta parte marca false ya que el operador == compara la ubicación en memoria del contenido, ya que estos no son iguales marca false

→ al usar equals, se compara el tipo de contenido, si se parecen, marcara true.

Este codigo si funciona, dando como salida false/true.

R3 Modelado : base, reto, y justificación

Cada multimetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar (persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre valido.

Base. Declara al menos dos variables (atributos) de instancia con tipo, con static y una constante final. Completa constructor y dos metodos: cambio y consulta.

Reto: ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Que dato nunca cambia?

Justifica. Una linea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 lineas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

```
public class Multimetro{
private final inventario;
private String responsable;
private static int prestados=0;

public Multimetro(String inventario){
  this.inventario = inventario;
  this.responsable = null;
}
public void prestar(String persona){
this.responsable = persona;
 prestados ++;
}
public String getResponsable(){
  return this.responsable;
  }
}
```

inventario de tipo string, instancia y final. Identifica cada aparato físico y su identificador no cambia una vez creado.

responsable tipo string, instancia y mutable. Pertenece individualmente a cada multimetro y que cambia de valor al prestarse.

El acumulador global comportido por todos los multimetros para llevar la cuenta que se incrementa con cada prestamo.

El constructor que recibe el número de inventario para analizar la constante de la instancia y que define a responsable en null ya que nace sin responsable.

El metodo que presta el multimetro y que le asigna un nuevo responsable a la instancia actual y actualiza el contador global estatico.

Metodo que retorna el nombre de la persona a cargo del multimetro consultado sin alterar el estado del objeto.

R4. Codigo de repetición.

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el numero a texto.

```
static int digitos(int folio){
    if (folio ==0){
        return 1;
    }
    int contador =0;
    while (folio >0){
     folio = folio/10;
      contador ++;
    }
    return contador;
}
```

Para el ejemplo 4072/10

Ya que el folio no es cero, por lo que va a entrar en el ciclo

1ro = 4072/10 = 407, contado = 1

2do = 407/10 = 40, contador = 2

3do = 40/10 = 4, contador = 3

4to = 4/10 = 0, contador = 4.

Ejemplo 0

Ya que el ejemplo nos da el valor de 0, se cumple la condición del if. Por lo que inmediatamente retorna el valor de 1.

Para el ejemplo 8:

Ya que el valor es distinto de cero, se inicia el ciclo while

1ra = 8/10 = 0, contador = 1.

Finaliza el ciclo.

R5. Codigo con selección

Clasifica la carga de un equipo. Usa un switch clasico sobre carga /10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos.

```
static String nivelCarga(int carga) {
    if (carga <0 || carga >50){
        return "INVALIDA";
    }
    switch (carga/10) {
     case 0;
     case 1;
      return "BAJA";
     case 2;
     case 3;
     return "MEDIA";
    case 4;
    case 5;
      return "ALTA";
    default:
     return "INVALIDA";
    }
}
```

EJEMPLO: Fuera del rango.

Si la carga es menor que 0 o mayor a 50.

La condición if (carga <0 || carga >50) se ve incompleta por lo que retorna de inmediato "INVALIDA".

De 0 a 19

Al realizar la división carga/10, si es un valor de 0-9 returna un valor 0 y entre 10-19 produce 1. Ambos returns regresan "BAJA".

De 20-39

La division carga/10, da 20-30 da un valor de 2 y si es de 30-39 da como resultado 3.

Ambos valores devuelven "MEDIA".

De 40-50

La división para el rango de 40 a 49 resulta un valor de 4 y de 50 da un valor de 5.

Ambos valores devuelven "ALTA".